

Growth Lead — Assessment Brief

Role: Product Vision Owner · Ordering & Payment | Format: Take-home · 5–7 days | Two parts: Technical + Business

This brief has two parts. **Part A** tests whether you can ship a working, AI-assisted system end-to-end on real infrastructure. **Part B** tests how you think about the business you would own. We are interested in **judgement, trade-offs and execution**, not perfect code or perfect slides. Please use Claude Code as your primary build agent for Part A — that is part of what we are evaluating.

Part A · Technical & AI Capability

Objective. Stand up an event-driven decisioning pipeline on AWS, built end-to-end using Claude Code. The domain is Ordering & Payment. The whole thing must run inside the AWS Free Tier.

What to build

- Create a personal **AWS Free Tier** account dedicated to this exercise.
- Generate a **dummy dataset of $\geq 100,000$ records** reflecting an ordering / payment context — e.g. orders, customers, SKUs, payment attempts. Realism matters; include skew, edge cases and a small share of fraud-shaped anomalies.
- Land the data in AWS using a store of your choice (DynamoDB, S3 + Athena, RDS). **Justify the choice** on cost, latency and the access pattern.
- Build a **trigger** that fires on new/updated records (DynamoDB Streams, S3 Event Notifications or EventBridge → Lambda) and runs a **multi-tier decision**: for example *Auto-approve · Manual review · Decline*, or *Fulfil from DC · Fulfil from store · Reject*.
- The decision must combine **at least three independent considerations** (e.g. basket value, customer tenure/LTV, real-time inventory, fraud score, payment-method risk). Persist the decision back to the store with full lineage.
- Build a **second, follow-up trigger** that reads the written decision and takes a downstream action — escalate, retry on a different rail, route to an alternative fulfilment node, or close out. This second step must read *and* write.
- Drive the whole build through **Claude Code**. We want to see the session, the prompts and the diffs — not just the final repo.

Deliverables

- AWS console access — a **read-only IAM user** with credentials shared via a secure channel (1Password link or similar). Do not share root credentials.
- Public GitHub repo with IaC (CDK, SAM or Terraform), source, seed script and a **README** covering architecture, cost envelope and how to tear it down.
- A **5–7 minute Loom** walking us through the architecture, one live trigger firing end-to-end, and what you would do differently with more time.
- One-page **architecture diagram** (PNG or draw.io).

How we score Part A

Architecture trade-offs · Quality of Claude Code orchestration (prompting, review, iteration) · Cost discipline (stays in Free Tier) · IaC and code hygiene · Observability and failure handling · Realism of dummy data · Clarity of the Loom narrative.

Part B · Business Acumen — Prioritisation

Context. Our Ordering business is run against four levers: **Assortment** (what we sell), **Availability** (whether it is in stock when the customer wants it), **Affordability** (price and perceived value vs. alternatives), and **DiFOT** (Delivery in Full, On Time). They are interlinked, finite resources compete across them, and they do not all move on the same time horizon.

What we want from you

Assume you start as Growth Lead on Monday. You have one quarter, one engineering pod, and one analyst. Answer the following in a written response (max 1 page) plus a single priority matrix slide:

1 · The call

- Which **one lever** would you fix first, and why? State your hypothesis in one sentence before you defend it.
- What is the **second-order effect** on the other three levers — what improves, what regresses, what is unblocked?

2 · The data you would need

Tell us what data would make this call *defensible rather than instinctive*, and where you would source it. Cover, at minimum:

- **Demand-side:** search-to-cart, cart-to-checkout, substitution behaviour, out-of-stock click-throughs, returning vs. new mix.
- **Supply-side:** in-stock rate by SKU and node, lead times, supplier fill rate, picker accuracy, last-mile DiFOT by route.
- **Competitive / market:** price index vs. top three competitors on the top-decile basket; assortment gaps weighted by demand.
- **Unit economics:** contribution margin per order, cost-to-serve by fulfilment path, discount depth vs. incremental volume.
- **Customer behaviour:** repeat rate, NPS by failure mode (wrong item, late, missing), churn cohorts after a single DiFOT failure.

3 · Sequencing & trade-offs

- Give one concrete example of how fixing your chosen lever **compromises** another (e.g. broader assortment hurts availability; deeper affordability hurts margin).
- How would you know within 30 days that you picked wrong? Name the leading indicator and the kill-criterion.

How we score Part B

Clarity of reasoning over volume of analysis · Willingness to call a decision under ambiguity · Honest acknowledgement of trade-offs (cost vs. growth, short vs. long term) · A clean line from *question* → *data* → *decision* → *kill-criterion*.

Logistics

Time window	5–7 calendar days from receipt of this brief.
Tools	Claude Code is required for Part A. Free choice for Part B.
Cost	Stay within AWS Free Tier. We do not reimburse cloud spend.
Submission	Single email with: AWS IAM creds, GitHub link, Loom link, 1-page PDF for Part B, architecture diagram.
Confidentiality	Reviewed by the hiring panel only. Tear the AWS account down after the final round.
Questions	Reply to this email — we will respond within one working day.